College Lost & Found Portal: Detailed Workflow

1. Introduction

This document outlines the detailed workflow for a College Lost & Found Portal, a web application designed to facilitate the reporting, claiming, and management of lost and found items within a college campus. The system will leverage a robust technology stack including HTML, CSS, JavaScript, Bootstrap for the frontend, Spring Boot for the backend, Hibernate for Object-Relational Mapping (ORM), MySQL as the relational database, and JSP for server-side rendering of dynamic web pages.

2. System Architecture Overview

The College Lost & Found Portal will follow a multi-tiered architecture, typically a Model-View-Controller (MVC) pattern, to ensure separation of concerns, scalability, and maintainability.

- **Presentation Layer (Frontend):** This layer will be responsible for user interaction and displaying information. It will be built using HTML, CSS, JavaScript, and Bootstrap for responsive design and enhanced user experience. JSP will be used to generate dynamic content on the server-side before sending it to the client's browser.
- **Application Layer (Backend):** Developed with Spring Boot, this layer will handle business logic, process user requests, interact with the database, and manage security. It will expose RESTful APIs for communication with the frontend.
- **Persistence Layer:** Hibernate, an ORM framework, will manage the mapping between Java objects and the relational database. This abstracts away direct SQL interactions, making data access more efficient and less error-prone.
- **Database Layer:** MySQL will serve as the primary data store, housing all information related to users, lost items, found items, categories, and claim requests.

3. Core Features and Workflow

3.1 User Management

Objective: To allow students to register, log in, and manage their profiles securely.

Workflow:

1. **Registration:**

- **Frontend (HTML, CSS, JS, Bootstrap, JSP):** Users access a registration form. Client-side validation (JavaScript) ensures basic data integrity (e.g., email format, password strength).
- **Backend (Spring Boot):** Upon form submission, the Spring Boot controller receives the user data. A service layer handles business logic, including checking for existing email addresses and hashing the password (e.g., using BCrypt). An OTP (One-Time Password) is generated and sent to the user's college email for verification.
- **Persistence (Hibernate & MySQL):** User details, including the hashed password and a flag for `is_verified` (initially false), are stored in the `users` table. The OTP and its expiration are stored in a temporary table or cache.
- **Email Service:** Spring Boot integrates with an email service to send the OTP.

2. **Email Verification (OTP):**
    - **Frontend:** User receives an email with an OTP and is prompted to enter it on a verification page.
    - **Backend:** The entered OTP is validated against the stored OTP. If valid, the `is_verified` flag for the user in the `users` table is set to true.
3. **Login:**
    - **Frontend:** Users enter their credentials (email, password) on the login page.
    - **Backend:** Spring Security (integrated with Spring Boot) authenticates the user by comparing the provided password with the hashed password in the database. Upon successful authentication, a session is established.
4. **Password Reset:**
    - **Frontend:** User requests a password reset, providing their email.
    - **Backend:** A password reset token is generated and sent to the user's email. This token is stored in the `password_reset_tokens` table with an expiration time.
    - **Frontend:** User clicks on the reset link in the email, which directs them to a page to set a new password, along with the token.
    - **Backend:** The token is validated. If valid and not expired, the user's password in the `users` table is updated with the new hashed password.

3.2 Item Management (Lost/Found Items)

Objective: To enable users to post lost or found items with descriptions and images, and to view/filter existing items.

Workflow:

1. **Add Lost Item:**

- **Frontend:** User navigates to

  an "Add Lost Item" page. They fill out a form including item name, description, category, and upload an image.
- **Backend:** The Spring Boot controller receives the item details and the image file. The image is stored in a designated directory on the server or an external storage service (e.g., AWS S3). The item details, including the image file path and `status` as 'lost', are saved to the `items` table via Hibernate.

2. **Add Found Item:**
   - **Frontend:** Similar to adding a lost item, users navigate to an "Add Found Item" page, providing details and an image.
   - **Backend:** The process is identical to adding a lost item, but the `status` is set to 'found' in the `items` table.
3. **View and Filter Items:**
   - **Frontend:** Users can view a list of all lost or found items. Filters (e.g., by category, status, date posted) and search functionality are available.
   - **Backend:** Spring Boot controllers, using Hibernate, retrieve items from the `items` table based on filter criteria and search queries. The data is then passed to JSP pages for rendering.

3.3 Claim Request System

Objective: To allow users to request to claim a found item and for administrators to verify and approve these claims.

Workflow:

1. **Initiate Claim:**
   - **Frontend:** A user viewing a found item can click a "Claim Item" button. They are prompted to provide details proving ownership (e.g., specific characteristics of the item, where/when it was lost).
   - **Backend:** The claim request, along with the user's provided details, is stored in a new `claims` table (or an extension of the `items` table with claim-specific fields). The `claimed` flag in the `items` table is set to `true` and the `status` is updated to 'pending verification'.
2. **Admin Verification:**
   - **Admin Panel (Frontend/JSP):** Administrators access a dedicated panel to view pending claim requests. Each request displays the item details, the claimant's information, and their proof of ownership.

- **Admin Panel (Backend):** Spring Boot controllers retrieve pending claims. Administrators can review the details and decide to approve or reject the claim.

3. **Status Update:**
    - **Admin Action:** If the admin approves the claim, the `status` of the item in the `items` table is updated to 'returned'. If rejected, the `claimed` flag is reset to `false` and the `status` is reverted to 'found'.
    - **User Notification:** The system can send an email notification to the claimant regarding the status of their claim.

3.4 Admin Verification Panel

Objective: To provide administrators with tools to manage items, verify claims, and oversee user activity.

Workflow:

1. **Login:** Admins log in through a dedicated admin login page.
2. **Dashboard:** Upon successful login, admins are presented with a dashboard showing an overview of new lost/found items, pending claims, and user statistics.
3. **Item Management:** Admins can view, edit, or delete any lost or found item. They can also manually change the status of an item.
4. **Claim Management:** As described in Section 3.3, admins can review and act on claim requests.
5. **User Management:** Admins can view user profiles, activate/deactivate accounts, and potentially reset passwords.

4. Technology Stack Breakdown

- **Frontend:**
    - **HTML5:** Structure of web pages.
    - **CSS3:** Styling of web pages, often enhanced with preprocessors like SASS/LESS if desired.
    - **JavaScript (ES6+):** Client-side interactivity, form validation, AJAX calls for dynamic content updates.
    - **Bootstrap:** Responsive design framework for consistent UI across devices.
    - **JSP (JavaServer Pages):** Server-side templating technology to embed Java code into HTML, generating dynamic web content.
- **Backend:**

    - **Spring Boot:** Framework for building robust, production-ready Spring applications with minimal configuration. Provides embedded servers (Tomcat), auto-configuration, and a vast ecosystem of libraries.
    - **Spring MVC:** Part of Spring Framework, used for building web applications following the MVC pattern. Handles request mapping, view resolution, and data binding.
    - **Spring Security:** Provides authentication and authorization services for the application.
- **Persistence:**
    - **Hibernate (JPA Implementation):** Object-Relational Mapping (ORM) framework that maps Java objects to database tables and vice-versa. Simplifies database interactions.
    - **Spring Data JPA:** Provides an abstraction layer over JPA, further simplifying data access operations with repositories.
- **Database:**
    - **MySQL:** A popular open-source relational database management system for storing structured data.

5. Development Process (High-Level)

1. **Database Design:** Define the schema for `users`, `categories`, `items`, `claims`, and `password_reset_tokens` tables in MySQL.
2. **Backend Development (Spring Boot, Hibernate):**
    - Set up Spring Boot project.
    - Define JPA entities (e.g., `User`, `Category`, `Item`, `Claim`) corresponding to database tables.
    - Create Spring Data JPA repositories for CRUD operations.
    - Develop service layers to encapsulate business logic.
    - Implement RESTful API endpoints using Spring MVC controllers for frontend communication.
    - Integrate Spring Security for authentication and authorization.
    - Implement email service for OTP and notifications.
3. **Frontend Development (HTML, CSS, JS, Bootstrap, JSP):**
    - Design and implement user interfaces for registration, login, item posting, item viewing, claim submission, and admin panel using HTML, CSS, and Bootstrap.

- Use JavaScript for client-side validation and dynamic UI elements.
- Utilize JSP to render dynamic data from the backend and create interactive forms.

4. **Deployment:** Deploy the Spring Boot application to a server (e.g., Tomcat, embedded).

6. Conclusion

This detailed workflow provides a comprehensive understanding of the College Lost & Found Portal project, from user interactions to backend processing and database management. By following this structured approach and leveraging the chosen technologies, a robust and user-friendly application can be developed to efficiently manage lost and found items within a college environment.